Multi-Agent AI Engineering Curriculum

A self-study curriculum for building production-grade multi-agent AI systems — 20 documents, 13 core projects, 3 real-world example projects, and 16 practice twins, from Python fundamentals to a deployed, tested, multi-agent platform.

This is not a theory course. Every document pairs its concepts with hands-on practice exercises (each with progressive hints and full worked solutions), and every project is real, runnable code you build yourself, step by step.

How this curriculum is structured

- **20 documents** (`01_python_foundations/` → `20_web_frontends/`) — each one teaches a topic through simple-English "Core Concepts," then a set of practice exercises with a 2-hint, 3-depth (Basic/Intermediate/Advanced) hint system and full solutions, then a Build Task that feeds directly into the projects below.
- **13 projects** (`project_1_supportdesk_chat_and_triage/` → `project_13_codeguard_pr_review/`) — real multi-agent systems you build across the documents, growing from a 2-agent terminal app to a tested, deployed, production-ready platform — each with a **practice twin** (a second build in a new story; see below).
- Every document and project is delivered as both a `README.md` (source) and a matching `README.pdf` (for comfortable reading), with fully clickable internal links between exercises, hints, and solutions.

Documents

#	Document	Feeds into
01	Python & Engineering Foundations	—
02	APIs, HTTP, JSON & Backend Basics	—
03	LLM Basics	—
04	OpenAI API	Project 1
05	LangChain Basics	—
06	Tools & Function Calling	Projects 7, 8 (MCP topics)
07	AI Agents	Project 2

#	Document	Feeds into
08	RAG	—
—	Async Python (checkpoint)	—
09	LangGraph	—
10	Agent Workflows	Project 3
11	Multi-Agent Systems	Project 4
12	Production AI Engineering	—
13	Testing, Evaluation & Observability	—
14	Debugging Lab	—
15	Five Projects (reference index)	—
16	System Design & Architecture	—
17	Interview Preparation	—
18	Final Production Multi-Agent Capstone	Project 5
19	MLOps & LLMOps	—
20	Web Frontends for AI Apps (Streamlit, Flask, FastAPI, Django)	Any project's UI

Projects

#	Project	Agents	Built during	Proves
1	SupportDesk: Concierge & Triage	2	Docs 01-04	You can talk to a model correctly and safely, and route between two simple agents
2	ResearchHand: Worker & Verifier	2	Docs 05-07	You can give a model real abilities, check its choices, and add an independent check
3	DocuMind: Retriever, Reasoner & Approval	3	Docs 08-10	You can build agents as a clear, checkable graph, with real handoffs
4	ContentForge: Supervisor-Led Team	5	Doc 11	You can split a problem across specialist agents, with a supervisor routing on the fly
5	ContentForge Pro: Production Platform	5 (production-ready)	Docs 12-19	You can ship, test, secure, watch, version, and defend the design of

#	Project	Agents	Built during	Proves
				#4
6 (bonus)	LangChainPro: Production LCEL Patterns	—	Doc05	You can build a real multi-step pipeline (branching, parallel calls, retries/fallbacks) in LangChain alone, without needing LangGraph
7 (bonus)	MCPForge: MCP Server	— (server, not agents)	Doc06's MCP topics	You can expose real tools/resources/prompts over the Model Context Protocol, usable by any MCP client
8 (bonus)	MCPBridge: MCP Client Agent	1 (dynamic tools)	Doc06/07's MCP + ReAct topics	You can build an agent whose tools are discovered at runtime from live MCP servers, not hardcoded
9 (bonus)	PromptShield: RAG Agent Hardened Against Prompt Injection	1	Doc06/08's prompt injection topics	You can defend a RAG agent against injected instructions, and prove it with a real red-team test suite
10 (bonus)	MemoryKeeper: Persistent Assistant That Remembers You Across Sessions	1	Doc09's long-term memory topic	You can give an agent real memory that survives across separate conversations, with a working "forget me"
11 (bonus)	MCPCrew: Multi-Agent Research Team Powered by MCP Servers	3 (+ Supervisor)	Doc06/11's MCP + multi-agent topics	You can build a Supervisor-led team where each specialist connects to its own MCP server, with graceful degradation if one goes down
12 (bonus)	MCPResearch: Multi-Agent Pipeline Exposed as One MCP Tool	2 (Researcher + Fact-Checker)	Doc06/11's MCP + multi-agent topics	You can wrap an entire agentic pipeline behind a single MCP tool call — the "agent-as-a-service" pattern
13 (bonus)	CodeGuard: PR-Review Team	4	All of Docs 01-19, applied again	You can apply the whole architecture to a new problem, with less help

Agent count grows deliberately across the arc (2 → 2 → 3 → 5 → 5) — each project adds one new idea on top of the last, not several at once.

Real-World Example Projects

Beyond the numbered arc, these are standalone **real-world example projects** — each a complete, step-by-step build of a job a real business has, reusing the exact skills from the documents (document loaders, structured output, RAG, multi-agent, validation). Each one also has a practice twin (below).

#	Project	Skills	Builds a
14	InvoiceIQ — PDF Invoice Data Extraction & Validation	Document loaders, structured output, Pydantic validation	Pipeline that extracts and *checks* invoice data
15	MeetingMind — Transcript to Decisions, Actions & Summary	Multi-agent, `RunnableParallel`, structured output	Team of parallel extractors + a compiler
16	JobMatch — Resume to Job-Description Skill Matcher	RAG (semantic matching), structured output	Resume scorer that matches by meaning, not keywords

Practice Twins

Each project has a **practice twin** — the same skills in a brand-new real-world story, delivered as a story only (no step-by-step walkthrough) plus hints and a complete worked solution. You build it yourself from the story, and check the hints or solution only if you get stuck. They're optional but are the fastest way to prove a pattern really stuck.

Mirrors	Practice twin	Build it from
Project 1	RepairDesk — Device-Repair Chat & Ticket-Intake Assistant	its story + solution
Project 2	LoanDesk — Tool-Using Finance Assistant With a Verifier	its story + solution
Project 3	PolicyPal — HR Policy Assistant With Citations & Approval	its story + solution
Project 4	TripCraft — Multi-Agent Travel Itinerary Generator	its story + solution
Project 5	TripCraft Pro — Production Itinerary Service	its story + solution
Project 6	CampusDesk — Student-Helpdesk Ticket-Triage Pipeline	its story + solution
Project 7	LinkKeeper — Personal Bookmarks MCP Server	its story + solution
Project 8	OpsPilot — On-Call Assistant Over Discovered MCP Tools	its story + solution
Project 9	ReviewGuard — Injection-Hardened Product-Review Summarizer	its story + solution
Project 10	FitPal — Persistent Fitness Coach With Cross-Session Memory	its story + solution
Project 11	ClientLens — Multi-Agent Customer-360 Team Over MCP Services	its story + solution
Project 12	ComplyDesk — Policy-Checked Support Replies Behind One MCP Tool	its story + solution

Mirrors	Practice twin	Build it from
Project 13	DispatchGuard — Multi-Agent Outgoing-Message Review in One Pass	its story + solution
Project 14	ReceiptRadar — Expense-Receipt Extraction & Policy Check	its story + solution
Project 15	FeedbackLens — Customer-Feedback Batch Analyzer	its story + solution
Project 16	CourseFit — Course Prerequisite Readiness Checker	its story + solution

How to use this

1. Start at `01_python_foundations/` and work through the documents in order — each one lists its prerequisites at the top.
2. Read a document's Core Concepts, then do its Practice Exercises. Try each one yourself before opening its hints — use Hint 1, then Hint 2 only if you're still stuck, and the full solution only as a last resort.
3. When a document's Build Task is done, check `15_five_projects_index/` and start the project it unlocks.
4. See `CURRICULUM.md` for the full syllabus, rationale, and document-by-document breakdown.

Building the PDFs

Every `README.md` has a matching `README.pdf` with clickable internal links. If you edit any `.md` file, regenerate its PDF with the build tool in `tools/`:

```
cd tools
python3 build_pdfs.py                    # rebuild everything
python3 build_pdfs.py ../01_python_foundations/README.md   # just one file
python3 check_links.py                   # verify every internal link resolves
```

`watch_pdfs.py` can also be left running to rebuild automatically whenever a `.md` file changes.